Análise de Desempenho e Otimização em Arquiteturas Paralelas: Um Estudo de Caso para Redes Neurais Convolucionais

Leonardo César Cota de Castro[1], **Bernardo Borges Machado**[1]

[1]Departamento de Ciência da Computação – Universidade Federal de Minas Gerais (UFMG)
Belo Horizonte – MG – Brasil

{leonardo.cota, bernardo.machado}@dcc.ufmg.br

Abstract. *Matrix multiplication (GEMM) is the computational core of Convolutional Neural Network (CNN) training. This work investigates the architectural causes for the acceleration of this operation on GPUs. Starting from a profiling analysis, CUDA C++ kernels were progressively implemented with different optimization levels (Tiling, Thread Coarsening), comparing them against CPU execution (Serial/OpenMP) and proprietary libraries (cuBLAS). Results demonstrate how efficient memory hierarchy management and specialized hardware (Tensor Cores) usage are determinants for Deep Learning performance.*

Resumo. *A multiplicação de matrizes (GEMM) é o núcleo computacional do treinamento de Redes Neurais Convolucionais (CNNs). Este trabalho investiga as causas arquiteturais da aceleração dessa operação em GPUs. Partindo de uma análise de profiling, implementou-se progressivamente kernels em CUDA C++ com diferentes níveis de otimização (Tiling, Thread Coarsening), comparando-os com execuções em CPU (Serial/OpenMP) e bibliotecas proprietárias (cuBLAS). Os resultados demonstram como o gerenciamento eficiente da hierarquia de memória e o uso de hardware especializado (Tensor Cores) são determinantes para o desempenho em Deep Learning.*

1. Introdução

A computação de alto desempenho (HPC) é vital para avanços científicos e industriais, tendo na multiplicação de matrizes (GEMM) seu núcleo computacional. Com o advento das Redes Neurais Convolucionais (CNNs), a demanda por GEMM cresceu exponencialmente, visto que a operação de convolução é computacionalmente traduzida como multiplicação de matrizes pelo método *im2col* [Chetlur et al. 2014]. Assim, a otimização deste *kernel* impacta diretamente a eficiência em Deep Learning e outras aplicações numéricas.

Este trabalho apresenta um estudo comparativo e incremental sobre a implementação de algoritmos de multiplicação de matrizes. O objetivo é dissecar o desempenho computacional partindo de uma execução serial em CPU, passando por paralelismo em múltiplos núcleos (OpenMP), até atingir implementações massivamente paralelas em Unidades de Processamento Gráfico (GPU) utilizando a arquitetura CUDA. O estudo culmina na comparação com bibliotecas proprietárias otimizadas (cuBLAS), servindo como um levantamento técnico para guiar implementações futuras.

2. Fundamentação Teórica e Motivação

2.1. O Gargalo Computacional em CNNs

A análise inicial do projeto baseou-se em implementações de alto nível de Redes Neurais em Python utilizando o framework TensorFlow. A motivação prática para este estudo de otimização surgiu da observação direta do desempenho durante o treinamento de uma CNN para reconhecimento de CAPTCHAs.

Ao executar o treinamento do modelo por 30 épocas fixas, registrou-se um tempo total de **55m 47s** utilizando processamento em CPU. Em contraste, a mesma carga de trabalho foi concluída em apenas **3m 48s** ao utilizar a aceleração da GPU NVIDIA RTX 4070. Este resultado representa um *speedup* prático de aproximadamente **14,7 vezes** para a aplicação completa.

Para compreender a natureza deste ganho e os limites da performance, realizou-se o *profiling* do código utilizando a ferramenta **NVIDIA Nsight Systems**. A análise identificou que a maior parte do tempo de execução é consumida em operações de álgebra linear densa, especificamente *kernels* de multiplicação de matrizes (`sgemm`) e convoluções.

Visto que a complexidade algorítmica de uma multiplicação de matrizes ingênua de dimensão $N \times N$ é de $O(N^3)$, para valores de N na ordem de milhares (comuns tanto nas camadas densas quanto nas convoluções transformadas via *im2col*), o custo computacional em arquiteturas sequenciais torna-se proibitivo. Assim, a eficiência destes *kernels* específicos é o fator determinante para o desempenho global do sistema.

2.2. Arquiteturas Paralelas: CPU vs. GPU

A CPU é projetada para baixa latência e execução de threads complexas e sequenciais. Em contrapartida, a GPU adota uma arquitetura *throughput-oriented*. Dispositivos modernos da NVIDIA, baseados na arquitetura CUDA, possuem milhares de núcleos de processamento simplificados. Isso permite que a GPU execute o modelo SIMT (*Single Instruction, Multiple Thread*), onde milhares de threads executam a mesma instrução sobre dados diferentes simultaneamente, cenário ideal para a independência de dados inerente à multiplicação de matrizes.

3. Metodologia

A metodologia adotada consistiu no desenvolvimento progressivo de seis abordagens de processamento para realizar a operação $C = A \times B$, onde A, B e C são matrizes quadradas de dimensão N. Para os testes finais de estresse e análise de escalabilidade, utilizou-se $N = 8192$ (totalizando aprox. 67 milhões de elementos), garantindo uma carga de trabalho suficiente para saturar a largura de banda de memória.

O ambiente de desenvolvimento utilizou a linguagem C++ para o controle de fluxo, a API OpenMP para paralelismo em CPU e a plataforma NVIDIA CUDA para computação em GPU.

O hardware utilizado consistiu em uma **GPU** NVIDIA GeForce RTX 4070 (Arquitetura Ada Lovelace, 46 SMs, 184 Tensor Cores) e uma **CPU** AMD Ryzen 7 5800X3D (8 núcleos, 16 threads).

Foram implementadas as seguintes abordagens:

1. **CPU Serial:** Implementação de referência.
2. **CPU Paralela:** Utilização de múltiplos núcleos via OpenMP [Pacheco 2011].
3. **GPU Ingênua:** Implementação direta da lógica matemática em CUDA [Kirk and Hwu 2012]..
4. **GPU Otimizada (Tiled):** Uso de memória compartilhada para reduzir latência.
5. **GPU Otimizada (Coarsening):** Aumento da granularidade da thread para reuso de registradores.
6. **GPU cuBLAS:** Utilização da biblioteca de álgebra linear da NVIDIA.

4. Implementação e Otimização

A estratégia de implementação seguiu uma abordagem incremental, partindo de uma versão serial básica até kernels altamente otimizados para a arquitetura de GPU. O objetivo foi identificar e mitigar os gargalos de desempenho em cada estágio, focando principalmente na hierarquia de memória e na intensidade aritmética.

4.1. Processamento em CPU

A implementação serial de referência percorre as linhas da matriz A e colunas da matriz B através de três laços aninhados (*triple loop*) para calcular o produto escalar. Embora algoritimamente correta ($O(N^3)$), essa abordagem é ineficiente em processadores modernos pois não explora a vetorização (SIMD) nem o paralelismo de múltiplos núcleos.

A versão otimizada via OpenMP introduziu o paralelismo de *threads*, utilizando a diretiva `#pragma omp parallel for` para distribuir as iterações do laço externo entre os núcleos físicos disponíveis. Apesar do ganho de desempenho significativo com o uso de múltiplos núcleos, essa abordagem rapidamente atinge o limite da largura de banda da memória RAM (*Memory Wall*). Para matrizes grandes ($N = 8192$), os dados não cabem no cache L3, forçando a CPU a buscar dados na memória principal repetidamente, deixando as unidades de processamento ociosas.

4.2. GPU: Abordagem Ingênua (Naive)

A primeira implementação em CUDA decompõe o problema atribuindo a responsabilidade de calcular um único elemento da matriz resultante C para uma única *thread*. O kernel utiliza índices globais bidimensionais para mapear a thread (x, y) à coordenada (col, row) da matriz.

Embora elimine os laços externos e ofereça paralelismo massivo, esta abordagem sofre de uma ineficiência crítica no acesso à memória: a **redundância de leitura**. Para calcular uma linha inteira da matriz C, a mesma linha da matriz A precisa ser lida da Memória Global (VRAM) N vezes (uma para cada coluna de B). Como a VRAM possui alta latência (centenas de ciclos de clock) e largura de banda limitada comparada aos registradores, o desempenho é severamente restringido pelo tráfego de memória (*memory bound*).

4.3. GPU: Otimização via Tiling e Memória Compartilhada

Para solucionar a redundância de acesso à VRAM, implementou-se a técnica de *Tiling* (Ladrilhamento). O princípio fundamental é explorar a **localidade temporal** dos dados.

O kernel divide as matrizes de entrada e saída em sub-blocos quadrados de dimensão 32×32. Esses blocos são dimensionados para caber na **Memória Compartilhada** (*Shared Memory*) de cada Streaming Multiprocessor (SM) da GPU. A execução ocorre em etapas cooperativas:

1. As threads de um bloco cooperam para carregar um pequeno pedaço (tile) de A e B da lenta Memória Global para a rápida Memória Compartilhada.
2. Executa-se uma barreira de sincronização (`__syncthreads()`) para garantir a integridade dos dados.
3. As threads calculam o produto parcial lendo exclusivamente da Memória Compartilhada, que possui latência próxima à dos registradores.
4. O processo se repete para o próximo tile.

Essa estratégia reduz o tráfego de memória global por um fator proporcional ao tamanho do bloco (no caso, 32x), movendo o gargalo da largura de banda da VRAM para a capacidade de computação dos núcleos.

4.4. GPU: Otimização via Thread Coarsening

Buscando maximizar a intensidade aritmética, aplicou-se a técnica de *Thread Coarsening* (Aumento de Granularidade). Na versão *Tiled* padrão, cada thread calcula apenas 1 elemento de saída. Nesta versão otimizada, cada thread é responsável por calcular **2 elementos adjacentes** da matriz C (ex: $C[y][x]$ e $C[y][x+1]$).

A vantagem técnica reside no **reuso de registradores**. Para calcular dois elementos vizinhos horizontalmente, é necessário acessar a mesma linha da matriz A. Em vez de duas threads lerem o mesmo valor da memória compartilhada, uma única thread lê o valor de A uma vez, armazena-o em um registrador local, e o utiliza para realizar duas operações de multiplicação-soma (FMA) distintas com os valores correspondentes de B.

Isso resulta em:

- Redução de 50% no número de leituras da matriz A na memória compartilhada.
- Redução do overhead de instruções de controle de laço e indexação (*instruction-level parallelism*).
- Maior trabalho útil por thread, escondendo melhor a latência de memória.

4.5. O Estado da Arte: cuBLAS

Finalmente, utilizou-se a biblioteca `cublasSgemm` como baseline de desempenho máximo. Esta biblioteca implementa otimizações de baixo nível inacessíveis em CUDA C++ padrão, como *assembly* manual (SASS), *software pipelining* (para carregar dados do próximo bloco enquanto calcula o atual) e, crucialmente, o uso de **Tensor Cores**. Diferente dos núcleos CUDA escalares usados em nossas implementações manuais, os Tensor Cores executam operações matriciais em bloco, com tamanhos que variam conforme o tipo numérico (por exemplo, $16 \times 16 \times 16$ em FP16/BF16 na Ada Lovelace) , atingindo throughputs ordens de magnitude superiores.

5. Análise de Resultados

Os experimentos foram conduzidos na GPU RTX 4070 (184 Tensor Cores). A Tabela 1 sumariza os tempos obtidos para as quatro dimensões de teste, evidenciando a escalabilidade das soluções.

Tabela 1. Tempos de execução (em segundos) para diferentes dimensões de matriz N.

Implementação	N=1024	N=2048	N=4096	N=8192
1. CPU Serial	4.3279	43.0394	354.8920	3371.5800
2. CPU OpenMP	0.3504	3.9472	35.4904	353.7130
3. GPU Ingênua	0.0013	0.0583	0.0810	0.5815
4. GPU Tiled	0.0010	0.0348	0.0607	0.4850
5. GPU Coarsened	0.0005	0.0152	0.0282	0.2276
6. GPU cuBLAS	0.0002	0.0031	0.0061	0.0534

5.1. Comparativo Geral de Desempenho

A Figura 5.1 apresenta a visão macro do desempenho. À esquerda (a), o tempo em escala logarítmica evidencia o abismo entre as arquiteturas. À direita (b), o speedup absoluto confirma que a aceleração atinge escalas de $63.000\times$ para matrizes grandes.

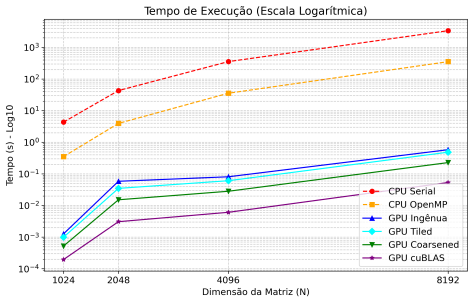


Figura 1. Tempo de execução (Log).

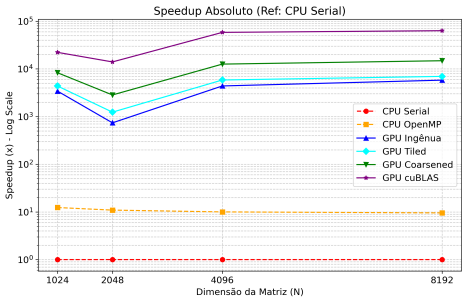


Figura 2. Speedup vs. CPU Serial.

Isso mostra que o openMP tem um speedup consistente com relação à versão serial. Contudo, esse ganho se torna irrelevante perto da GPU.

5.2. Impacto das Otimizações Manuais em GPU

Focando apenas nas implementações de GPU, a Figura 5.2 detalha o impacto da engenharia de software. À esquerda (a), a escala linear mostra a redução absoluta de tempo. À direita (b), o speedup relativo isola o ganho das otimizações manuais.

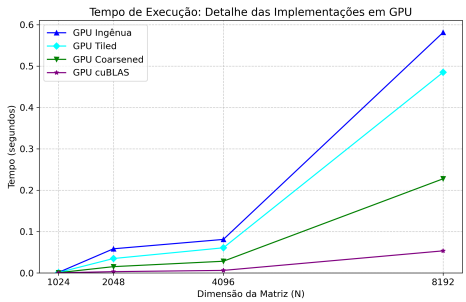


Figura 3. Tempo GPU (Linear).

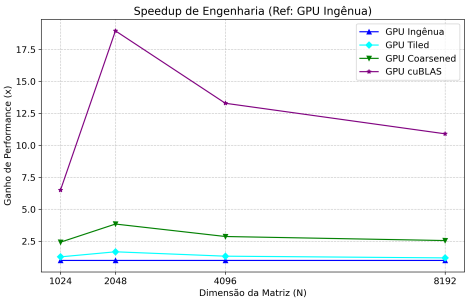


Figura 4. Speedup de Engenharia.

A implementação **GPU Coarsened** (linha verde) demonstrou consistência superior, atingindo um ganho entre $2.5\times$ e $3.8\times$ sobre a versão ingênua, validando a eficácia do reuso de registradores. Já a curva da **cuBLAS** revela o salto proporcionado pelos Tensor Cores e as demais otimizações da NVIDIA.

6. Discussão Arquitetural

A diferença massiva entre CPU e GPU Ingênua ilustra a distinção entre arquiteturas focadas em latência (CPU) e em vazão (GPU). Para operações densas, a capacidade da GPU de lançar milhares de threads esconde a latência de memória. Ao analisar as otimizações manuais (Ingênua $\rightarrow$ Tiled $\rightarrow$ Coarsened), fica evidente o impacto da hierarquia de memória: o Tiling moveu o gargalo da VRAM para a Shared Memory, e o Coarsening otimizou o uso de registradores.

6.1. Discrepância entre Speedup Sintético e Real

Nota-se uma diferença entre o speedup obtido no benchmark sintético (laboratório) e o prático na CNN. No benchmark, isolamos a computação pura, atingindo milhares de vezes de aceleração. Na aplicação real (CNN), fatores como a Lei de Amdahl, overhead de transferência de dados via PCIe e latência de lançamento de múltiplos kernels diluem o ganho total, resultando em um speedup prático de cerca de 14x.

6.2. Estratégias Avançadas de Otimização na cuBLAS

A eficiência obtida pela cuBLAS não se deve apenas ao hardware, mas a uma estrutura de software que espelha a hierarquia física da GPU. A biblioteca implementa um aninhamento de *loops* hierárquico que alinha blocos de threads, *warps* e threads individuais com a hierarquia de memória (Global $\rightarrow$ Compartilhada $\rightarrow$ Registradores), maximizando a localidade dos dados.

No nível mais alto, a matriz é dividida em grandes *tiles* processados por blocos de threads. Dentro destes, ocorre uma subdivisão para o nível de *Warp*, onde grupos de threads movem dados da memória compartilhada para os registradores. Para evitar que as unidades de cálculo fiquem ociosas durante essas transferências, utiliza-se uma técnica agressiva de *Software Pipelining* com *Double Buffering*. O sistema mantém o processador ocupado com o cálculo do bloco atual (nos registradores) enquanto, simultaneamente, pré-carrega os dados do próximo bloco em um espaço reservado (*buffer*) na memória compartilhada. Essa latência é mascarada pelo paralelismo de instruções.

Além disso, uma vez que o resultado parcial de $C = A \times B$ reside nos registradores (que são o nível mais rápido de memória), a biblioteca realiza operações extras (como fusão de bias ou ativação ReLU) diretamente nestes registradores antes de escrever o resultado final na memória global, economizando largura de banda.

Para cenários onde as dimensões da matriz não são ideais (ex: matrizes muito ”magras”ou pequenas), a cuBLAS aplica estratégias de redução paralela conhecidas como **Split-K**. Isso permite particionar a dimensão da soma (o laço K) entre múltiplos blocos de threads, aumentando o paralelismo e evitando que a GPU fique subutilizada. Nas arquiteturas mais recentes (Hopper/Ada Lovelace), essa lógica evolui para a **Especialização de Warps**: um grupo de *warps* dedica-se exclusivamente a mover dados (usando a unidade de Cópia Assíncrona), enquanto outro grupo foca apenas na matemática dos Tensor

Cores, operando como produtores e consumidores coordenados via barreiras de hardware (*mbarrier*).

6.3. Otimização de Acesso à Memória: Tile Swizzling

Uma discrepância notável nos resultados experimentais ocorreu na dimensão $N = 2048$, onde todas as implementações manuais (Ingênua, Tiled e Coarsened) apresentaram uma degradação de eficiência relativa, fenômeno não observado na biblioteca cuBLAS.

Esta degradação é atribuída ao *Partition Camping*. Como a largura da matriz (2048×4 bytes) é uma potência de 2, o padrão linear de acesso aos blocos faz com que múltiplas *threads* acessem endereços mapeados para o mesmo canal físico de memória DRAM simultaneamente, criando congestionamento enquanto outros canais permanecem ociosos.

A NVIDIA contorna este problema implementando Tile Swizzling (ou *Block Rasterization*). Em vez de processar os blocos da matriz na ordem linear padrão (linha por linha), a biblioteca altera dinamicamente a ordem de lançamento dos blocos (seguindo determinados padrões). Isso garante que, a qualquer momento, os blocos em execução estejam distribuídos espacialmente pela matriz, espalhando a carga de leitura uniformemente entre todos os controladores de memória da GPU e evitando a saturação de canais específicos.

6.4. Seleção Dinâmica de Algoritmos (Autotuning)

Uma discrepância observada nos experimentos foi a latência elevada na primeira execução da função `cublasSgemm`, seguida por um desempenho ótimo nas chamadas subsequentes. Este comportamento é atribuído aos mecanismos de **Autotuning** da biblioteca.

Diferentemente da nossa implementação manual, que utiliza uma estratégia estática de paralelização (32×32), a cuBLAS possui um repositório de *kernels* distintos (baseados em GEMM, Winograd ou FFT). Durante a primeira chamada, a biblioteca realiza um *profiling* em tempo de execução, testando candidatos para as dimensões específicas ($N = 8192$) e arquitetura da GPU. O algoritmo vencedor é armazenado em cache, garantindo que as execuções futuras utilizem a estratégia ótima sem o *overhead* de decisão.

7. Estudo de Tensor Cores

Sob uma perspectiva de arquitetura VLSI, um Tensor Core é uma unidade funcional especializada (introduzida na arquitetura Volta e aprimorada na Ada Lovelace [NVIDIA Corporation 2024]) para executar operações matriciais em bloco. Embora os Tensor Cores originais operassem sobre matrizes 4×4, nas arquiteturas mais recentes (incluindo a Ada Lovelace utilizada neste trabalho) o tamanho efetivo da operação depende do tipo numérico, com tiles típicos de $16 \times 16 \times 16$ (FP16/BF16) e $8 \times 8 \times 16$ (TF32). Cada instrução disparada por um warp executa dezenas de operações FMA internas em uma única operação matricial ($D = A \cdot B + C$), resultando em um *throughput* muito superior ao dos CUDA Cores.

O desafio arquitetural reside em manter esses blocos alimentados: a aritmética envolve compressores e realinhamento de ponto flutuante, e os Tensor Cores ocupam área

significativa dentro do SM, contendo unidades *Fused Multiply Add* (FMA) organizadas para baixa latência (cerca de quatro ciclos). Mais importante que a latência é a capacidade de aceitar uma nova operação matricial a cada *clock*, desde que haja dados disponíveis.

O gargalo real é justamente essa alimentação. Para sustentar o *throughput*, a GPU utiliza técnicas como *double buffering* e pipeline interno, garantindo que blocos de dados estejam prontos no momento correto. Ao eliminar arredondamentos intermediários e utilizar multiplicação–soma em cascata (CMA), o pipeline consegue empilhar operações, disfarçando a latência interna.

Em resumo, embora cada operação leve alguns ciclos para ser concluída, o Tensor Core inicia uma nova operação por ciclo quando plenamente alimentado. Essa engenharia (envolvendo caminhos de dados paralelos e organização densa de unidades) é o que torna inviável replicar tal eficiência utilizando apenas CUDA Cores convencionais, explicando o abismo de desempenho observado entre nossa implementação manual e a biblioteca cuBLAS.

8. Conclusão e Trabalhos Futuros

Este estudo percorreu o caminho crítico da otimização para hardware paralelo. Demonstrou-se que, para problemas de álgebra linear densa, a simples migração para GPU oferece muitos ganhos, mas é a reestruturação algorítmica (memória compartilhada) que desbloqueia o verdadeiro potencial do hardware. Conclui-se que o limite físico imposto pelos núcleos CUDA convencionais é superado apenas pelo uso de unidades especializadas como os Tensor Cores.

Como expansão natural deste projeto, sugere-se: a implementação manual de kernels utilizando instruções intrínsecas de Tensor Cores (WMMA API) e a exploração de precisão mista (FP16) para reduzir o uso de memória sem perda significativa de acurácia.

Referências

Chetlur, S., Woolley, C., Vandermersch, P., Cohen, J., Tran, J., Catanzaro, B., and Shelhamer, E. (2014). cudnn: Efficient primitives for deep learning. *arXiv preprint arXiv:1410.0759*.

Kirk, D. B. and Hwu, W.-m. W. (2012). *Programming Massively Parallel Processors: A Hands-on Approach*. Morgan Kaufmann, 2nd edition.

NVIDIA Corporation (2024). *CUDA C++ Programming Guide*. NVIDIA Corporation. Release 12.4.

Pacheco, P. S. (2011). *An Introduction to Parallel Programming*. Morgan Kaufmann.